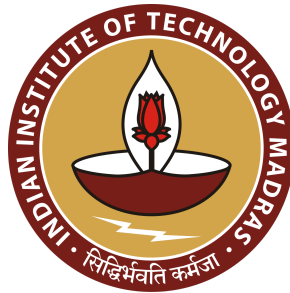


GRAND CHALLENGE - 2025

Verilog IP Integration with Shakti SoC



DEVELOPED BY :
SHAKTI DEVELOPMENT TEAM @ IITM
SHAKTI.ORG.IN

0.1 Proprietary Notice	3
0.2 Release Information	4
Introduction	5
Integration of IP supports AXI4	5
Integration of IP supports AXI4 Lite	9
Steps to add external interrupts from Verilog IP	13
Steps to include the Verilog files and run	15

0.1 Proprietary Notice

Copyright © 2025, **SHAKTI@IIT Madras**. All rights reserved.
Information in this document is provided “as is,” with every effort ensuring that the documentation is as accurate as possible.

SHAKTI@IIT Madras expressly disclaims all warranties, representations, and conditions of any kind, whether express or implied, including, but not limited to, the implied warranties or conditions of merchantability, fitness for a particular purpose and non-infringement.

SHAKTI@IIT Madras does not assume any liability rising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation indirect, incidental, special, exemplary, or consequential damages.

SHAKTI@IIT Madras reserves the right to make changes without further notice to any products herein.

The project was funded by Ministry of Electronics and Information Technology (MeitY), Government of India

0.2 Release Information

Version	Date	Updates	Reviewed by
1.0	16/09/2025	Vignesh K	Kotteeswaran E
Draft V.1	07/10/2025	Poovarasan M	Kotteeswaran E

Introduction

This document details the step by step procedure to integrate the **Custom Verilog IP** with Shakti. Any IP that supports AXI4/AXI4 Lite interface will be integrated at fpga_top level. IP should have the AXI4/AXI4 Lite slave interface which will be integrated with the AXI4/AXI4 Lite Master interface of Shakti SoC.

The files specified in this document are present in the gc2025/hw/boards/art7_ganga/ directory. Compare the images with the original content of files to observe the changes, most of the changes will be newly added lines, only some lines will be modified. The same steps will be applicable to the integration of custom Verilog IP with Yamuna SoC also, but the changes alone should be made in gc2025/hw/boards/art7_yamuna/ directory.

To integrate the IP operates at higher frequency AXI4 interface [Fast Fabric] will be used, whereas for IP operates at lower frequency AXI4 Lite interface [Slow Fabric] can be used.

This document will have following four sections, the third and fourth section will be common to first and second,

1. Integration of IP supports AXI4.
2. Integration of IP supports AXI4 Lite.
3. Steps to add external interrupts from Verilog IP.
4. Steps to include the verilog files and run.

Note: The highlighted lines in the following images will indicate the changes. The line numbers shown in the images might not match exactly.

Integration of IP supports AXI4

File: "gc2025/hw/boards/art7_ganga/Soc.defines"

Step 1: Increment the "Num_Fast_Slaves" by 1

```
26  `ifdef simulate
27  `define Num_Fast_Slaves 8
28  `else
29  `define Num_Fast_Slaves 7
30  `endif
31  `define Memory_slave_num 0
32  `define Clint_slave_num 1
33  `define Debug_slave_num 2
34  `define FastErr_slave_num 3
35  `define Slow_fabric_slave_num 4
36  `define Boot_slave_num 5
37  `ifdef simulate
38  `define Sign_slave_num 6
39  `endif
```

Step 2: Add “``define AXI4_slave_num n`” (replace the *n* by the last slave number - 1) and increment the `Sign_slave_num` number by 1.

```

26  `ifdef simulate
27  |   `define Num_Fast_Slaves 8
28  |   `else
29  |   `define Num_Fast_Slaves 7
30  |   `endif
31  `define Memory_slave_num 0
32  `define Clint_slave_num 1
33  `define Debug_slave_num 2
34  `define FastErr_slave_num 3
35  `define Slow_fabric_slave_num 4
36  `define Boot_slave_num 5
37  `define AXI4_slave_num 6
38  `ifdef simulate
39  |   `define Sign_slave_num 7
40  |   `endif

```

Step 3: Add “``define AXI4Base *addr1`” and “``define AXI4End *addr2`” . Define `*addr1` and `*addr2` such that it does not conflict with other slave addresses.

```

42  `define MemoryBase      'h8000_0000
43  `define MemoryEnd      'h8FFF_FFFF
44  `define TCMBBase      'h9000_0000
45  `define TCMEnd      'h9FFF_FFFF
46  `define ClintBase      'h0200_0000
47  `define ClintEnd      'h020B_FFFF
48  `define DebugBase      'h0000_0000
49  `define DebugEnd      'h0000_0FFF
50  `define BootBase      'h0000_1000
51  `define BootEnd      'h0000_8FFF
52  `define AXI4Base      'h0001_0000
53  `define AXI4End      'h0001_00FF
54  `define SlowBase      'h0001_1300
55  `define SlowEnd      'h0004_7FFF
56  `ifdef simulate
57  |   `define SignBase      'h0005_0000
58  |   `define SignEnd      'h0005_000c
59  |   `endif

```

These are the 3 steps that need to be done in the “Soc.defines” file.

File: "gc2025/hw/boards/artys_a7_ganga/Soc.bsv"

Step 1: Add the decoding logic for "AXI4_slave_num" selection.

```
74     function Bit#(TLog#(`Num_Fast_Slaves)) fn_slave_map_fast (Bit#(`paddr) addr);
75         Bit#(TLog#(`Num_Fast_Slaves)) slave_num = 0;
76         if(addr >= `MemoryBase && addr <= `MemoryEnd)
77             slave_num = `Memory_slave_num;
78         else if(addr >= `ClintBase && addr <= `ClintEnd)
79             slave_num = `Clint_slave_num;
80         `ifdef debug
81             else if(addr >= `DebugBase && addr <= `DebugEnd)
82                 slave_num = `Debug_slave_num;
83         `endif
84         else if(addr >= `BootBase && addr <= `BootEnd)
85             slave_num = `Boot_slave_num;
86         ✨ else if(addr >= `AXI4Base && addr <= `AXI4End)
87             ... slave_num = `AXI4_slave_num; You, 4 days ago • Uncommitted changes
88         else if(addr >= `SlowBase && addr <= `SlowEnd)
89             slave_num = `Slow_fabric_slave_num;
90         else if(addr >= `PLICBase && addr <= `PLICEnd)
91             slave_num = `Slow_fabric_slave_num;
92         `ifdef simulate
93             else if(addr >= `SignBase && addr <= `SignEnd)
94                 slave_num = `Sign_slave_num;
95         `endif
96         else
97             slave_num = `FastErr_slave_num;
98
99         return slave_num;
100     endfunction:fn_slave_map_fast
```

Step 2: Declare the "AXI4_Master_IFC" interface inside the "Ifc_chip_io" interface.

```
122     interface Ifc_chip_io;
123         interface Ifc_sspi_io spi0_io;
124         interface RS232 uart0_io;
125         method I2C_out i2c0_out; //I2c IO interface
126         (*always_ready, always_enabled*)
127         // interface AXI4_Lite_Master_IFC#(`paddr, 32, 0) xadc_master;
128         ✨ interface AXI4_Master_IFC#(`paddr, ELEN, 0) axi4_master; Yc
129         interface AXI4_Lite_Master_IFC#(`paddr, 32, 0) eth_master;
130         interface AXI4_Master_IFC#(`paddr, ELEN, 0) mem_master;
131         interface IOCellSide iocell_io;
```

Step 3: Connect the declared AXI4_Master_IFC with name “**axi4_master**” to the AXI4 fast fabric.

```
651 interface iocell_io = mixed_cluster.pinmuxtop_iocell_side;
652 // interface xadc_master = mixed_cluster.xadc_master;
653 ✨ interface axi4_master = fabric.v_to_slaves[AXI4_slave_num];
654 interface eth_master = slow_fabric.v_to_slaves[Eth_slave_num];
655 interface mem_master = fabric.v_to_slaves[Memory_slave_num];
656 method Action ext_interrupts(Bit#(2) i);
657 | wr_ext_interrupts <= i;
658 endmethod
659 endinterface;
660
661 endmodule: mkSoc
662 endpackage: Soc
```

File: “gc2025/hw/boards/artys_a7_ganga/fpga_top.v”

Step 1: Initialise the ports and wires that are required and part of your IP. Refer to the AXI ethernet lite IP of Xilinx as an example.

Step 2: In the mkDebugSoc instantiation, connect the wires from your IP’s AXI4 slave interface to the AXI4 Master interface (**axi4_master**) of Shakti SoC.

```
594 ✨ // AXI4 Master interface to be connected to Verilog IP
595 ..... .axi4_master_AWVALID(),
596 ..... .axi4_master_AWADDR(),
597 ..... .axi4_master_AWPROT(),
598 ..... .axi4_master_AWLEN(),
599 ..... .axi4_master_AWSIZE(),
600 ..... .axi4_master_AWBURST(),
601 ..... .axi4_master_AWID(),
602 ..... .axi4_master_WVALID(),
603 ..... .axi4_master_WDATA(),
604 ..... .axi4_master_WSTRB(),
605 ..... .axi4_master_WLAST(),
606 ..... .axi4_master_WID(),
607 ..... .axi4_master_BREADY(),
608 ..... .axi4_master_ARVALID(),
609 ..... .axi4_master_ARADDR(),
610 ..... .axi4_master_ARPROT(),
611 ..... .axi4_master_ARLEN(),
612 ..... .axi4_master_ARSIZE(),
613 ..... .axi4_master_ARBURST(),
614 ..... .axi4_master_ARID(),
615 ..... .axi4_master_RREADY(),
616 ..... .axi4_master_AWREADY(),
617 ..... .axi4_master_WREADY(),
618 ..... .axi4_master_BVALID(),
619 ..... .axi4_master_BRESP(),
620 ..... .axi4_master_BID(),
621 ..... .axi4_master_ARREADY(),
622 ..... .axi4_master_RVALID(),
623 ..... .axi4_master_RRESP(),
624 ..... .axi4_master_RDATA(),
625 ..... .axi4_master_RLAST(),
626 ..... .axi4_master_RID(),
```


File: "gc2025/hw/boards/arty_a7_ganga/constraints.xdc"

Step 1: Add the initialized ports in the fpga_top with the appropriate mapping in the "constraints.xdc" file.

Integration of IP supports AXI4 Lite

File: "gc2025/hw/boards/arty_a7_ganga/Soc.defines"

Step 1: Increment the "MixedCluster_Num_Slaves" by 1

```
123 // - Mixed Cluster
124 `define MixedCluster_Num_Slaves 10
125 `define MixedCluster_Num_Masters 1
126
```

Step 2: Add "``define AXI4Lite_slave_num n`" (replace the *n* by the last slave number - 1) and increment the MixedCluster_err_slave_num number by 1.

```
123 // - Mixed Cluster
124 `define MixedCluster_Num_Slaves 9
125 `define MixedCluster_Num_Masters 1
126
127 `define I2C0_slave_num 0
128 `define I2C1_slave_num 1
129 `define GPTimer0_slave_num 2
130 `define GPTimer1_slave_num 3
131 `define PLIC_slave_num 4
132 `define GPIO_slave_num 5
133 `define XADC_slave_num 6
134 `define Pinmux_slave_num 7
135 `define AXI4Lite_slave_num 8
136 `define MixedCluster_err_slave_num 9
```

Step 3: Add "``define AXI4LiteBase *addr1`" and "``define AXI4LiteEnd *addr2`". Define *addr1 and *addr2 such that it does not conflict with other slave addresses.

```
142 `define GPTimer0Base 'h0004_0200
143 `define GPTimer0End 'h0004_02FF
144 `define GPTimer1Base 'h0004_0300
145 `define GPTimer1End 'h0004_03FF
146 `define AXI4LiteBase 'h0004_0400
147 `define AXI4LiteEnd 'h0004_04FF
```

File: "gc2025/hw/boards/artys_a7_ganga/mixed_cluster.bsv"

Step 1: Add the decoding logic for "AXI4Lite_slave_num" selection.

```
104     function Bit#(TLog#('MixedCluster_Num_Slaves)) fn_slave_map (Bit#('paddr) addr);
105         Bit#(TLog#('MixedCluster_Num_Slaves)) slave_num = 0;
106         if(addr>= `GPIOBase && addr<= `GPIOEnd)
107             slave_num = `GPIO_slave_num;
108         else if(addr>= `PLICBase && addr<= `PLICEnd)
109             slave_num = `PLIC_slave_num;
110         // else if(addr>= `I2C1Base && addr<= `I2C1End)
111         //     slave_num = `I2C1_slave_num;
112         else if(addr>= `I2C0Base && addr<= `I2C0End)
113             slave_num = `I2C0_slave_num;
114         else if(addr>= `GPTimer0Base && addr<= `GPTimer0End)
115             slave_num = `GPTimer0_slave_num;
116         else if(addr>= `GPTimer1Base && addr<= `GPTimer1End)
117             slave_num = `GPTimer1_slave_num;
118         ✨ else if(addr>= `AXI4LiteBase && addr<= `AXI4LiteEnd)
119             → slave_num = `AXI4Lite_slave_num;
120         else if(addr >= `PinmuxBase && addr <= `PinmuxEnd)
121             slave_num = `Pinmux_slave_num;
122         else
123             slave_num = `MixedCluster_err_slave_num;
124
125         return slave_num;
126     endfunction:fn_slave_map
```

Step 2: Declare the "AXI4_Lite_Master_IFC" interface inside the "Ifc_mixed_cluster" interface.

```
49     interface Ifc_mixed_cluster;
50         ✨ interface AXI4_Lite_Master_IFC#('paddr, 32, 0) axi4lite_master;
51         method I2C_out i2c0_out; //I2c IO interface
52         // method I2C_out i2c1_out;
53         (*always_ready,always_enabled*)
54         interface Ifc_gptimer_io gptimer0_io;
```

Step 3: Connect the declared AXI4_Lite_Master_IFC with name "axi4lite_master" to the AXI4 Lite slow fabric.

```
191     interface pinmux_top_iocell_side = pinmux_top.pinmux_axi4lite_iocell_side;
192     interface pinmux_top_peripheral_side = pinmux_top.pinmux_axi4lite_peripheral_side;
193     interface slave = c2s_xactor.axi_side;
194     ✨ interface axi4lite_master = fabric.v_to_slaves[ `AXI4Lite_slave_num ];
195     // interface xadc_master = fabric.v_to_slaves[ `XADC_slave_num ];
196     endmodule
197 endpackage
198
```

File: "gc2025/hw/boards/art7_ganga/Soc.bsv"

Step 1: Declare the "AXI4_Lite_Master_IFC" interface inside the "Ifc_chip_io" interface.

```
122 interface Ifc_chip_io;
123     interface Ifc_ssbi_io spi0_io;
124     interface RS232 uart0_io;
125     ✨ interface AXI4_Lite_Master_IFC#(`paddr, 32, 0) axi4lite_master;
126     method I2C_out i2c0_out; //I2c IO interface
```

Step 2: Route the "axi4lite_master" interface from "mixed_cluster" to "Soc"

```
653 // interface xadc_master = mixed_cluster.xadc_master;
654 ✨ interface axi4lite_master = mixed_cluster.axi4lite_master;
655 interface axi4_master = fabric.v_to_slaves[`AXI4_slave_num];
656 interface eth_master = slow_fabric.v_to_slaves[`Eth_slave_num];
657 interface mem_master = fabric.v_to_slaves [`Memory_slave_num];
658 method Action ext_interrupts(Bit#(2) i);
659     wr_ext_interrupts <= i;
660 endmethod
661 endinterface;
662
663 endmodule: mkSoc
664 endpackage: Soc
```

File: "gc2025/hw/boards/art7_a7_ganga/fpga_top.v"

Step 1: Initialise the ports and wires that are required and part of your IP. Refer to the AXI ethernet lite IP of Xilinx as an example which is already integrated with Ganga and Yamuna SoC. If your ports are bidirectional (type: inout) use an [IO buffer](#) like the following [phy_mdio is the IO port].

```
897 //-----IOBUF FOR AXI Ethernet lite IP-----//
898 .IOBUF eth_mdio_inst(
899     .O(phy_mdio_i),
900     .IO(phy_mdio),
901     .I(phy_mdio_o),
902     .T(phy_mdio_t)
903 );
```

Step 2: In the mkDebugSoc instantiation, connect the wires from your IP's AXI4 Lite slave interface to the AXI4 Lite Master interface (**axi4lite_master**) of Shakti SoC.

```
594 // AXI4 Lite Master interface to be connected to Verilog IP
595 .axi4lite_master_awvalid(),
596 .axi4lite_master_awaddr (),
597 .axi4lite_master_awprot (),
598 .axi4lite_master_awsiz e (),
599 .axi4lite_master_wvalid (),
600 .axi4lite_master_wdata (),
601 .axi4lite_master_wstrb (),
602 .axi4lite_master_bready (),
603 .axi4lite_master_arvalid(),
604 .axi4lite_master_araddr (),
605 .axi4lite_master_arprot (),
606 .axi4lite_master_arsiz e (),
607 .axi4lite_master_rready (),
608 .axi4lite_master_m_awready_awready(),
609 .axi4lite_master_m_wready_wready (),
610 .axi4lite_master_m_bvalid_bvalid (),
611 .axi4lite_master_m_bvalid_bresp (),
612 .axi4lite_master_m_arready_arready(),
613 .axi4lite_master_m_rvalid_rvalid (),
614 .axi4lite_master_m_rvalid_rresp (),
615 .axi4lite_master_m_rvalid_rdata (),
616
```

File: "gc2025/hw/boards/art7_a7_ganga/constraints.xdc"

Step 1: Add the initialized ports in the fpga_top with the appropriate mapping in the "constraints.xdc" file.

Steps to add external interrupts from Verilog IP

Let assume the Verilog IP generates interrupt, which will be serviced by Shakti SoC through PLIC controller.

File: "gc2025/hw/boards/artty_a7_ganga/fpga_top.v"

Step 1: Increment the width of the wire named "interrupts" by 1.

```
212     wire                                s_axi_rlast;
213     wire                                s_axi_rvalid;
214     ✨ wire [11:0]                        device_temp;
215     wire [2:0] ..... interrupts;
216
```

Step 2: Pass the interrupt [for e.g. wire "verilog_ip_intr"] from the Verilog IP to the "interrupts" at the incremented width position.

```
897     wire verilog_ip_intr;
898     ✨ assign interrupts = {verilog_ip_intr, eth10_ip2intc_irpt, 1'b0};
899
```

File: "gc2025/hw/boards/artty_a7_ganga/Soc.bsv"

Step 1: Increment the width of the method "ext_interrupts" by 1 in declaration.

```
266     // method Bit#(1)gpio_31_outen;
267     ✨ (*always_ready, always_enabled*)
268     .. method Action ext_interrupts(Bit#(3) i);
269     .. endinterface
270
271     interface Ifc_soc_sb;
272     `ifdef rtl_dump
273     interface Sbread sbread;
274     method Maybe#(CommitLogPacket) commitlog;
275     `endif
```

Step 2: Increment the width of the wire "wr_ext_interrupts" by 1.

```
327     `ifdef simulate
328     Ifc_sign_dump signature <- mksign_dump();
329     `endif
330     ✨ Wire#(Bit#(3)) wr_ext_interrupts <- mkWire();
331
```

Step 3: Increment the width of the method “**ext_interrupts**” by 1 in instantiation.

```
658 method Action ext_interrupts(Bit#(3) i);
659 wr_ext_interrupts <= i;
660 endmethod
661 endinterface;
662
663 endmodule: mkSoc
664 endpackage: Soc
```

File: “gc2025/hw/boards/art7_ganga/mixed_cluster.bsv”

Step 1: Increment the width of the method “**interrupts**” by 1 in declaration.

```
61 interface PeripheralSide pinmux_top_peripheral_side;
62 (*always_ready, always_enabled*)
63 method Action interrupts(Bit#(14) inp);
```

Step 2: Increment the width of the wire “**wr_external_interrupts**” by 1.

```
149 Wire#(Bit#(14)) wr_external_interrupts <- mkDWire('d0);
150 Wire#(Bit#(2)) wr_sb_ext_interrupt <- mkDWire(0);
151
```

Step 3: Increment the width of the method “**interrupts**” by 1 in instantiation.

```
188 method Action interrupts(Bit#(14) inp);
189 wr_external_interrupts <= inp;
190 endmethod
```

Step 4: Increment the number of sources by 1 in the PLIC module “**mkplic**” instantiation.

```
91 (*synthesize*)
92 module mkplic(Ifc_plic_axi4lite#(`paddr, 32, 0, 36, 2, 7));
93 let ifc();
94 mkplic_axi4lite#(`PLICBase)_temp(ifc);
95 return ifc;
96 endmodule
```

Step 5: Increment the width by 1 in the “**plic_inputs**” which will be the sources for PLIC. The ordering is rearranged in the way that the existing Interrupts IDs assigned to the peripherals remain unchanged.

```
158 rule rl_connect_plic_connections;
159     let tmp <- gpio.sb_gpio_to_plic.get;
160     Bit#(16) lv_gpio_intr= truncate(pack(tmp));
161     ✨ Bit#(36) plic_inputs={wr_external_interrupts[11], wr_external_interrupts
162     [13:12], wr_external_interrupts[10:6], 1'b0, i2c0.isint, 2'b0, gptimer1.
163     sb_interrupt, gptimer0.sb_interrupt, lv_gpio_intr, wr_external_interrupts[5:0]};
162     plic.sb_frm_sources(plic_inputs);
163 endrule
```

Steps to include the Verilog files and run

Step 1: Create the directory “**ip_verilog**” to place all the verilog files of your IP inside that

```
$ git clone https://gitlab.com/shaktiproject/gc2025.git
$ cd gc2025/hw
$ mkdir ip_verilog
$ cp -r your_dir/* ip_verilog/
```

File: “gc2025/hw/Makefile”

Step 2: Add the following lines in the “**Makefile**”

```
181 .PHONY: generate_verilog
182 generate_verilog: $(BSVBUILDDIR)/$(TOP_BIN)
183     @cp ${BS_VERILOG_LIB}/../Verilog.Vivado/RegFile.v ${VERILOGDIR}
184     @cp ${BS_VERILOG_LIB}/../Verilog.Vivado/BRAM2BEOload.v ${VERILOGDIR}
185     ✨ @cp ./ip_verilog/* ${VERILOGDIR} You, 2 seconds ago • Uncommit
186     @cp ${BS_VERILOG_LIB}/FIF02.v ${VERILOGDIR}
187     @cp ${BS_VERILOG_LIB}/FIF01.v ${VERILOGDIR}
```

Step 3: Run the following command to generate the bitstream

For **Ganga** SoC,

```
$ make build BOARD=arty_a7_ganga
```

For **Yamuna** SoC,

```
$ make build BOARD=arty_a7_yamuna XLEN_build=32
```